

AGEX Knowledge Engine

1. 문서 개요

1.1 문서 목적

본 문서는 AGEX Knowledge Engine의 구조, 책임, 데이터 흐름, 수집·정규화·인덱싱·검색·재순위·인용·버전관리·권한관리·품질평가 및 Lifecycle을 정의한다.

AGEX에서 Knowledge는 단순한 문서 저장소가 아니다.

Knowledge는 Agent와 Workflow가 판단, 생성, 검증 및 실행에 사용할 수 있도록 출처, 구조, 권한, 버전, 신뢰도 및 검색 정책이 관리되는 공식 정보 자산이다.

본 문서는 다음 설계의 기준으로 사용한다.

- Knowledge Resource Model
- Knowledge Source
- Knowledge Collection
- Ingestion Pipeline
- Parsing
- Normalization
- Chunking
- Metadata
- Embedding
- Indexing
- Hybrid Retrieval
- Ranking
- Re-ranking
- Citation
- Knowledge Graph
- Structured Knowledge
- Retrieval Policy

- Access Control
- Versioning
- Freshness
- Provenance
- Quality Management
- Synchronization
- Deletion
- Governance
- Observability
- Security
- Cost Control

1.2 적용 범위

본 문서는 AGEX Knowledge Engine의 논리적 및 기술적 구조를 정의한다.

다음 세부 영역은 관련 문서와 연계한다.

- Agent Framework
- Workflow Engine
- Memory Engine
- Plugin Framework
- Runtime Architecture
- Security Architecture
- Multi-Tenant Architecture
- API Specification
- Governance
- SDK
- Deployment Architecture

1.3 Knowledge의 공식 정의

AGEX Knowledge는 Agent와 Workflow가 사용할 수 있도록 등록, 구조화, 검증, 인덱싱 및 통제된 정보 Resource이다.

Knowledge는 최소한 다음 특성을 가져야 한다.

- Source
- Ownership
- Tenant Scope
- Version
- Access Policy
- Data Classification
- Provenance
- Freshness
- Trust Level
- Retrieval Policy
- Lifecycle
- Citation Support

2. Knowledge Engine 목표

AGEX Knowledge Engine은 다음 목표를 충족해야 한다.

2.1 출처 추적 가능성

Agent가 사용한 정보가 어디에서 왔는지 추적할 수 있어야 한다.

2.2 신뢰 가능한 Retrieval

단순 벡터 유사도만으로 결과를 반환하지 않고 Query, Metadata, Keyword, Vector 및 정책을 종합하여 검색해야 한다.

2.3 Tenant 격리

다른 Tenant의 Knowledge가 검색 결과에 포함되지 않아야 한다.

2.4 모델 독립성

특정 Embedding Model이나 Vector Database에 종속되지 않아야 한다.

2.5 다양한 데이터 수용

정형·비정형·반정형 정보를 하나의 Knowledge 계층에서 사용할 수 있어야 한다.

2.6 지속적 갱신

원본 Source가 변경되면 Knowledge도 변경을 감지하고 다시 처리할 수 있어야 한다.

2.7 버전 관리

Knowledge 변경 이력을 추적하고 특정 Version 기준으로 조회할 수 있어야 한다.

2.8 품질 관리

부정확하거나 오래되거나 중복된 Knowledge를 탐지하고 관리할 수 있어야 한다.

2.9 정책 기반 접근

사용자와 Agent가 허용된 Knowledge Scope만 조회할 수 있어야 한다.

2.10 재사용성

하나의 Knowledge Collection을 여러 Agent와 Workflow에서 재사용할 수 있어야 한다.

3. Knowledge Engine 핵심 원칙

AGEX Knowledge Engine은 다음 원칙을 따른다.

- Knowledge as a First-Class Resource
- Provenance First
- Permission Before Retrieval
- Hybrid Retrieval
- Structured and Unstructured Support
- Model Agnostic Embedding
- Provider Agnostic Storage
- Versioned Knowledge

- Freshness Awareness
 - Citation by Default
 - Tenant Isolation
 - Incremental Indexing
 - Reproducible Retrieval
 - Quality Scoring
 - Explicit Retention
 - No Cross-Tenant Retrieval
 - No Unverified Silent Merge
 - No Embedding as Source of Truth
 - No Index as Source of Truth
 - Observable by Default
-

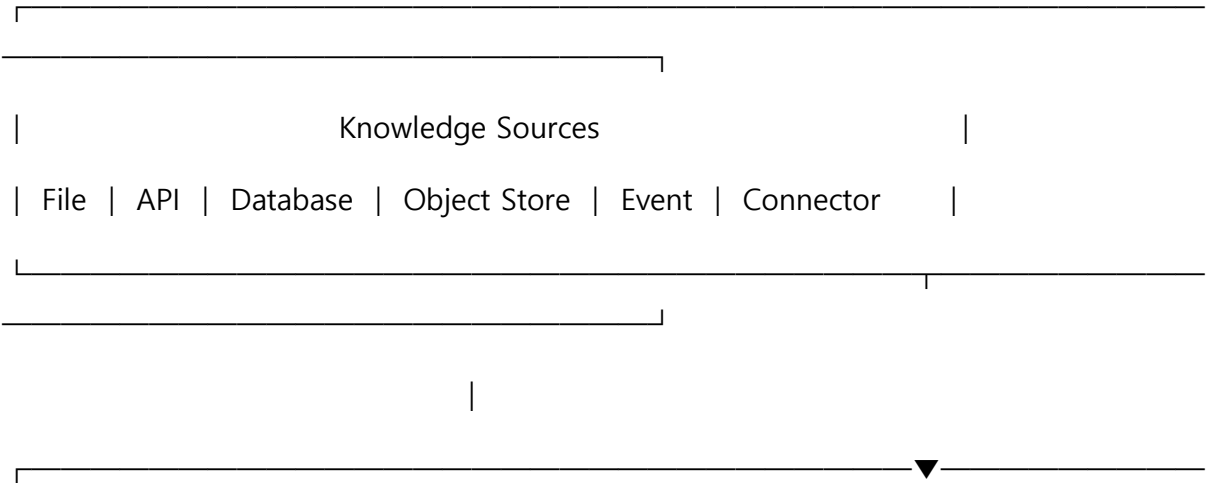
4. Knowledge Engine 전체 구조

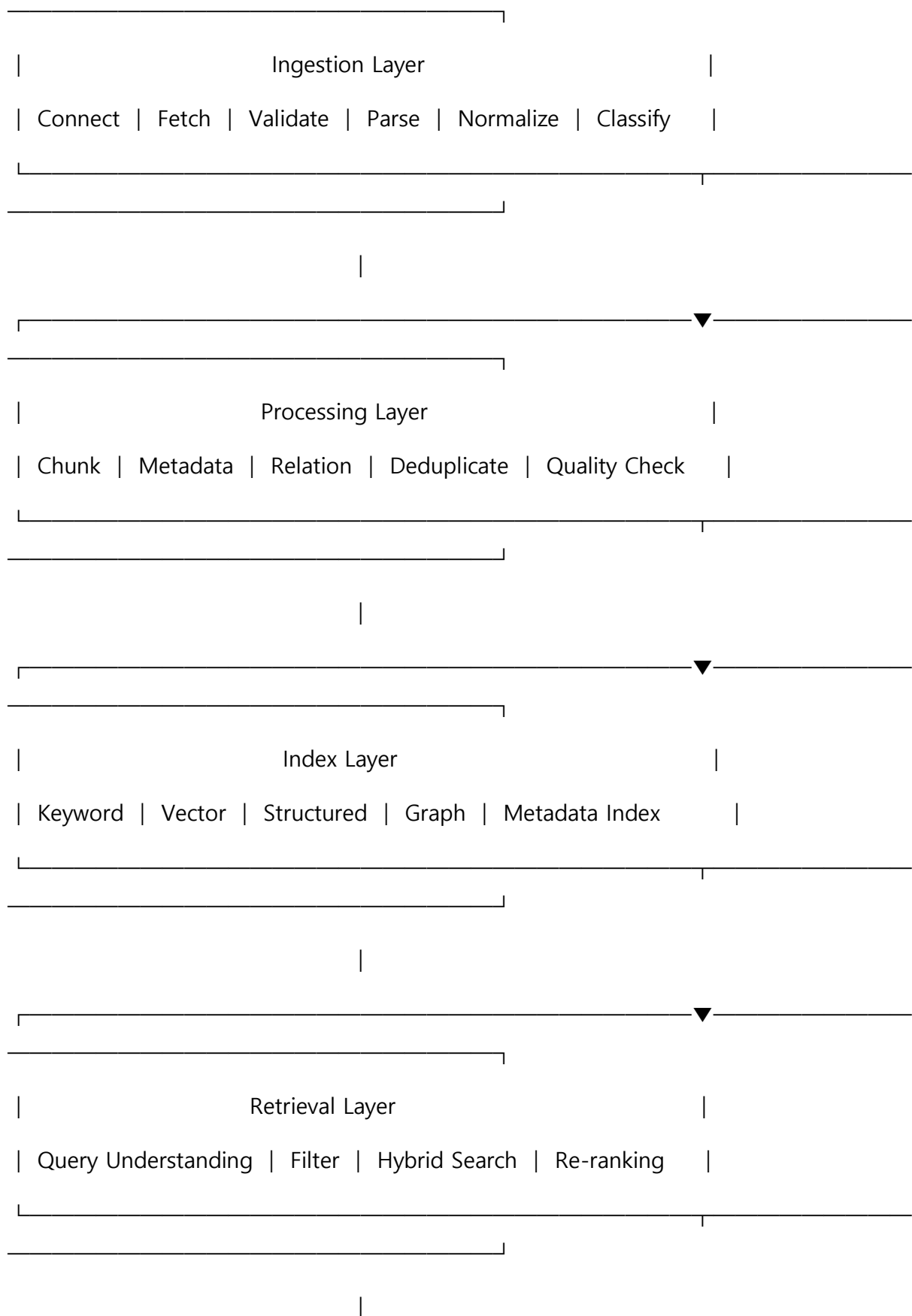
AGEX Knowledge Engine은 다음 주요 구성요소로 구성한다.

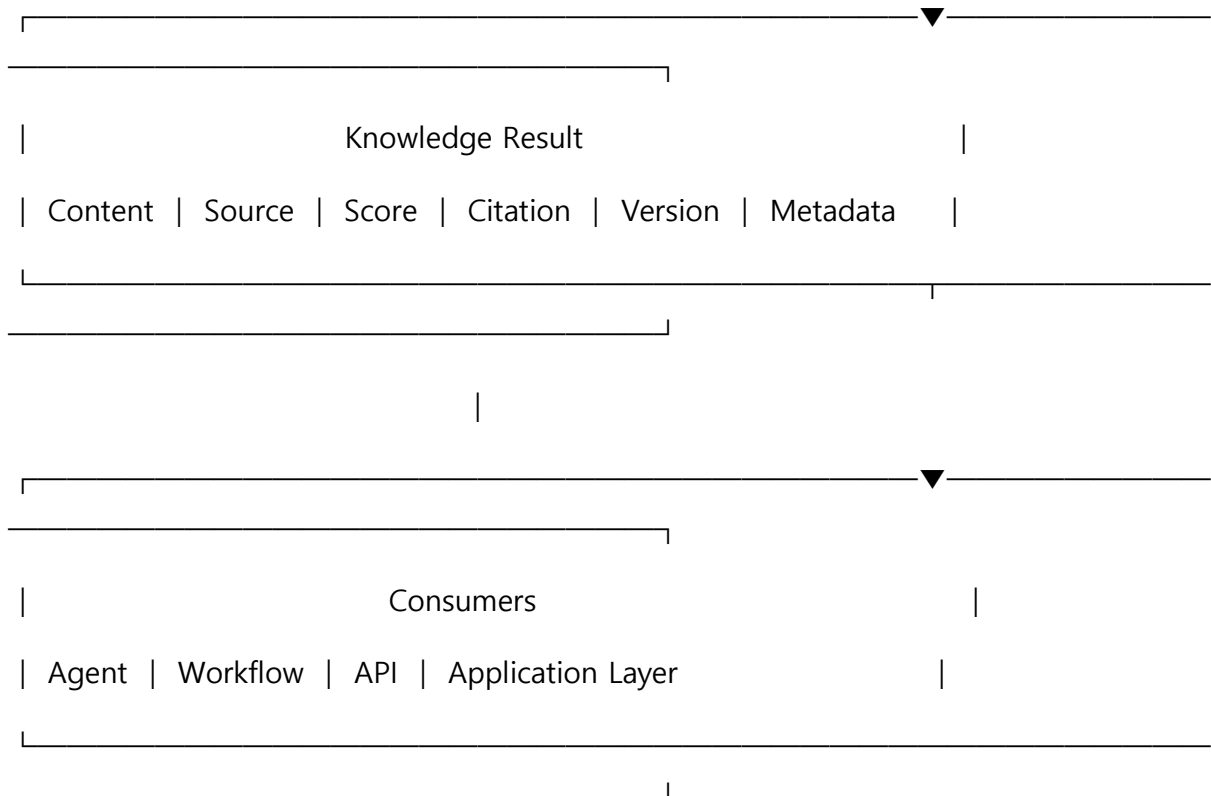
1. Knowledge Registry
2. Source Manager
3. Connector Manager
4. Ingestion Manager
5. Parser Framework
6. Normalization Engine
7. Classification Engine
8. Chunking Engine
9. Metadata Engine
10. Embedding Engine

- 11. Index Manager
- 12. Retrieval Engine
- 13. Query Understanding Engine
- 14. Hybrid Search Engine
- 15. Ranking Engine
- 16. Re-ranking Engine
- 17. Citation Engine
- 18. Knowledge Graph Engine
- 19. Structured Query Engine
- 20. Quality Engine
- 21. Freshness Manager
- 22. Synchronization Manager
- 23. Access Control Layer
- 24. Knowledge Lifecycle Manager
- 25. Knowledge Observability
- 26. Knowledge Storage Adapters

5. Knowledge Engine 개념 구조







모든 단계에 다음 공통 통제가 적용된다.

- Tenant
- Permission
- Policy
- Data Classification
- Audit
- Version
- Retention
- Observability

6. Knowledge Resource Model

6.1 Knowledge Resource 유형

AGEX는 Knowledge를 다음 Resource 유형으로 구분한다.

- Knowledge Source
- Knowledge Collection
- Knowledge Document
- Knowledge Item
- Knowledge Chunk
- Knowledge Entity
- Knowledge Relation
- Knowledge Index
- Knowledge Package

6.2 공통 속성

모든 주요 Knowledge Resource는 최소한 다음 속성을 가진다.

- Resource ID
- Tenant ID
- Namespace
- Resource Type
- Name
- Version
- Revision
- Source Reference
- Owner
- Classification
- Trust Level
- Access Policy
- Retention Policy
- Created At

- Updated At
- Effective From
- Effective Until
- Lifecycle State
- Metadata
- Checksum

6.3 Source of Truth

원본 Source 또는 검증된 Canonical Knowledge Record가 Source of Truth이다.

다음은 Source of Truth가 아니다.

- Vector Index
- Search Index
- Cache
- Embedding
- Retrieval Result
- Generated Summary

6.4 Derived Knowledge

원본에서 생성된 요약, Entity, Relation, Embedding 및 Chunk는 Derived Knowledge로 관리한다.

Derived Knowledge는 원본 Resource Reference를 반드시 유지해야 한다.

7. Knowledge Source

7.1 정의

Knowledge Source는 AGEX가 Knowledge를 수집하는 원본 위치 또는 데이터 공급자를 의미한다.

7.2 Source 유형

- File

- Object Storage
- Database
- API
- Event Stream
- Web Endpoint
- Repository
- External Knowledge Service
- User Input
- System Generated Data

7.3 Source 정보

Knowledge Source는 다음 정보를 가진다.

- Source ID
- Source Type
- Location
- Provider
- Connection Configuration
- Authentication Reference
- Synchronization Policy
- Parsing Policy
- Classification
- Retention
- Owner
- Trust Level
- Status

7.4 Source Trust Level

Source에는 신뢰 수준을 부여할 수 있다.

예:

- Authoritative
- Verified
- Trusted
- Unverified
- Restricted
- Deprecated

7.5 Source 상태

- Draft
- Connecting
- Active
- Degraded
- Suspended
- Disconnected
- Deprecated
- Archived

8. Knowledge Collection

8.1 정의

Knowledge Collection은 검색, 권한, 버전 및 운영을 함께 관리하기 위한 논리적 Knowledge 집합이다.

8.2 Collection 속성

- Collection ID
- Tenant ID

- Name
- Description
- Sources
- Retrieval Policy
- Embedding Policy
- Chunking Policy
- Access Policy
- Quality Policy
- Retention Policy
- Index Policy
- Metadata Schema
- Version
- Status

8.3 Collection 사용

Agent와 Workflow는 개별 Document가 아니라 Collection Scope를 기준으로 접근할 수 있다.

8.4 Collection 간 분리

Collection 간 데이터가 자동으로 혼합되어서는 안 된다.

Cross-Collection Retrieval은 명시적으로 허용된 경우에만 수행한다.

9. Knowledge Ingestion Pipeline

9.1 정의

Ingestion Pipeline은 원본 Source를 AGEX Knowledge Resource로 변환하는 처리 과정이다.

9.2 표준 흐름

Discover

- Connect
- Fetch
- Validate
- Classify
- Parse
- Normalize
- Deduplicate
- Segment
- Enrich Metadata
- Extract Entity
- Generate Embedding
- Index
- Verify
- Publish

9.3 Pipeline 실행 단위

Ingestion은 Workflow로 실행할 수 있어야 한다.

각 단계는 독립적 상태와 재시도 정책을 가진다.

9.4 Pipeline Idempotency

동일 Source Version을 반복 처리해도 중복 Knowledge가 생성되지 않아야 한다.

9.5 Partial Failure

일부 Document가 실패해도 전체 Collection을 자동 실패시키지 않는다.

실패 항목을 별도로 기록하고 재처리할 수 있어야 한다.

10. Source Discovery

10.1 정의

Source Discovery는 연결된 데이터 Source에서 처리할 신규 또는 변경 Resource를 탐색하는 과정이다.

10.2 탐지 방식

- Full Scan
- Incremental Scan
- Timestamp
- Version ID
- Checksum
- Event
- Webhook
- Change Data Capture

10.3 변경 식별

Source 변경 여부는 가능한 경우 Checksum 또는 Native Version을 사용한다.

단순 수정 시간에만 의존하지 않는 것을 권장한다.

11. Source Connector

11.1 정의

Connector는 외부 Source와 AGEX Knowledge Engine을 연결하는 표준 Adapter이다.

11.2 Connector Contract

Connector는 다음 Interface를 제공해야 한다.

- connect()
- healthCheck()
- listResources()
- fetchResource()
- getVersion()

- getMetadata()
- subscribeChanges()
- disconnect()

지원하지 않는 기능은 Capability로 명시한다.

11.3 Connector Manifest

- Connector ID
- Version
- Source Type
- Authentication Type
- Capabilities
- Configuration Schema
- Network Requirement
- Required Permission
- Rate Limit
- Runtime Compatibility

11.4 Connector 보안

Connector Credential은 Secret Store를 통해 관리한다.

Knowledge Parser 또는 Agent에 Credential을 전달하지 않는다.

12. Validation

12.1 입력 검증

수집된 Source는 처리 전에 다음을 검증한다.

- File Size
- MIME Type
- Encoding

- Schema
- Integrity
- Checksum
- Malware 여부
- Access Permission
- Tenant Ownership
- Data Classification

12.2 지원되지 않는 데이터

지원되지 않는 Format은 강제로 변환하지 않는다.

Unsupported 상태를 기록하고 Parser Extension을 요구한다.

12.3 손상 데이터

손상되거나 불완전한 Source는 정상 Knowledge로 Publish하지 않는다.

13. Parser Framework

13.1 정의

Parser Framework는 다양한 원본 형식을 표준 Knowledge Representation으로 변환한다.

13.2 Parser 유형

- Text Parser
- Document Parser
- Structured Data Parser
- HTML Parser
- JSON Parser
- XML Parser
- Tabular Parser
- Image Metadata Parser

- Audio Transcript Parser
- Custom Parser

13.3 Parser Output

Parser는 가능한 경우 다음 구조를 생성한다.

- Content
- Structure
- Heading
- Section
- Paragraph
- Table
- List
- Metadata
- Embedded Resource
- Source Location

13.4 Source Location 유지

원본 내 위치 정보를 가능한 한 유지한다.

예:

- Page
- Section
- Paragraph
- Row
- Column
- Timestamp
- Record Key

이는 Citation 생성에 사용된다.

14. Canonical Knowledge Representation

14.1 정의

서로 다른 Source 형식을 공통 내부 구조로 변환한다.

14.2 기본 구조

KnowledgeItem

- |— content
- |— content_type
- |— source
- |— source_location
- |— metadata
- |— classification
- |— version
- |— relations
- └— provenance

14.3 원본 보존

Normalization 과정에서 원본 의미를 임의로 변경하지 않는다.

원본 Content와 Normalized Content를 구분한다.

15. Normalization Engine

15.1 역할

Normalization Engine은 검색과 처리에 필요한 형식 정규화를 수행한다.

15.2 처리 가능 항목

- Encoding normalization
- Whitespace normalization

- Unicode normalization
- Structural cleanup
- Metadata normalization
- Date normalization
- Identifier normalization
- Language detection
- Duplicate marker removal

15.3 의미 변경 금지

Normalization은 사실 관계나 의미를 변경해서는 안 된다.

15.4 Generated Transformation

AI를 사용한 정규화나 재작성 결과는 원본과 별도의 Derived Resource로 저장해야 한다.

16. Data Classification

16.1 정의

Knowledge는 수집 단계에서 데이터 보안 등급을 가져야 한다.

16.2 Classification 적용 위치

- Source
- Document
- Chunk
- Entity
- Metadata
- Search Result

16.3 상속

하위 Resource는 기본적으로 상위 Source의 Classification을 상속한다.

더 높은 보안 등급을 부여하는 것은 가능하지만 임의로 낮출 수 없다.

16.4 Classification 기반 제어

Classification은 다음 결정에 사용한다.

- 저장 위치
 - Encryption
 - Model 사용 가능 여부
 - Retrieval 대상
 - 외부 Provider 전송
 - Retention
 - Audit 수준
-

17. Deduplication

17.1 정의

동일하거나 실질적으로 동일한 Knowledge가 반복 저장되는 것을 방지한다.

17.2 중복 탐지 방식

- Exact Checksum
- Source ID
- Record Key
- Text Fingerprint
- Semantic Similarity
- Metadata Match

17.3 Exact Duplicate

완전히 동일한 원본은 신규 Version 또는 Reference로 처리하고 불필요한 중복 인덱싱을 방지한다.

17.4 Semantic Duplicate

유사한 문서라고 자동 삭제하지 않는다.

중복 후보로 표시하고 정책에 따라 처리한다.

18. Chunking Engine

18.1 정의

Chunking Engine은 긴 Knowledge Resource를 Retrieval 가능한 단위로 분할한다.

18.2 Chunking 전략

- Fixed Token
- Fixed Character
- Paragraph
- Sentence
- Section
- Heading-Aware
- Semantic
- Recursive
- Table-Aware
- Code-Aware

18.3 Chunk Metadata

각 Chunk는 최소한 다음을 유지한다.

- Chunk ID
- Parent Resource ID
- Source ID
- Sequence
- Start Position
- End Position
- Heading

- Token Count
- Version
- Metadata
- Classification

18.4 Chunk Overlap

Overlap은 Retrieval 품질과 저장 비용을 고려해 정책으로 설정한다.

18.5 의미 단위 보호

표, 코드, 목록 또는 논리적 Section을 무의미하게 절단하지 않는 Chunking Strategy를 지원해야 한다.

19. Metadata Engine

19.1 Metadata 유형

- System Metadata
- Source Metadata
- User Metadata
- Extracted Metadata
- Classification Metadata
- Quality Metadata
- Lifecycle Metadata

19.2 주요 Metadata

- Author
- Created Date
- Modified Date
- Source
- Language

- Category
- Tags
- Effective Date
- Expiration Date
- Trust Level
- Security Classification
- Version
- Content Type

19.3 Metadata Schema

Collection별 Metadata Schema를 정의할 수 있어야 한다.

19.4 Metadata Search

Metadata는 Retrieval Filter에 사용할 수 있어야 한다.

20. Entity and Relation Extraction

20.1 정의

Knowledge에서 Entity와 Relation을 추출하여 구조화된 관계를 생성할 수 있다.

20.2 Entity 구조

- Entity ID
- Type
- Name
- Aliases
- Source
- Attributes
- Confidence
- Version

20.3 Relation 구조

- Relation ID
- Source Entity
- Target Entity
- Relation Type
- Source Reference
- Confidence
- Effective Time

20.4 검증

AI가 생성한 Entity와 Relation은 원본 Knowledge와 구분하여 Derived Data로 취급한다.

20.5 Confidence

낮은 Confidence의 관계는 자동 확정 Knowledge로 사용하지 않을 수 있다.

21. Knowledge Graph

21.1 정의

Knowledge Graph는 Entity와 Relation을 그래프 구조로 저장하고 탐색하는 선택적 Knowledge 표현 방식이다.

21.2 사용 목적

- Relationship Search
- Multi-Hop Retrieval
- Dependency Analysis
- Context Expansion
- Entity Resolution
- Provenance Tracking

21.3 Graph와 Vector의 관계

Knowledge Graph는 Vector Search를 대체하지 않는다.

Query 특성에 따라 함께 사용할 수 있어야 한다.

21.4 Graph Source

Graph의 모든 Node와 Edge는 가능한 한 원본 Knowledge Reference를 가져야 한다.

22. Embedding Engine

22.1 정의

Embedding Engine은 Knowledge Content를 Vector Representation으로 변환한다.

22.2 Model Abstraction

Embedding Model은 Provider Adapter를 통해 호출한다.

22.3 Embedding Policy

Collection마다 다음을 정의할 수 있다.

- Model
- Dimension
- Batch Size
- Normalization
- Language Policy
- Version
- Retry
- Cost Limit

22.4 Model 변경

Embedding Model이 변경되면 기존 Vector를 그대로 혼합하지 않는다.

새 Index Version을 생성하는 것을 원칙으로 한다.

22.5 Embedding Metadata

- Embedding Model

- Provider
- Model Version
- Dimension
- Created At
- Source Version
- Checksum

22.6 Embedding 보안

민감한 Knowledge를 외부 Embedding Provider로 전송할 수 있는지 Data Policy를 먼저 확인한다.

23. Index Manager

23.1 Index 유형

AGEX는 하나 이상의 Index를 조합할 수 있다.

- Keyword Index
- Vector Index
- Metadata Index
- Structured Index
- Graph Index
- Temporal Index

23.2 Index Resource

- Index ID
- Collection ID
- Type
- Version
- Provider

- Status
- Configuration
- Source Version
- Created At
- Last Updated At

23.3 Index Lifecycle

- Building
- Active
- Rebuilding
- Degraded
- Deprecated
- Archived

23.4 Index 재구성

Index는 Source of Truth에서 언제든지 재구성 가능해야 한다.

23.5 Zero-Downtime Reindex

대규모 Reindex는 신규 Index를 구축한 후 Alias 또는 Active Version을 전환하는 방식으로 수행할 수 있다.

24. Query Understanding Engine

24.1 정의

Query Understanding Engine은 사용자 또는 Agent Query를 검색 가능한 구조로 변환한다.

24.2 처리 항목

- Intent Detection
- Query Normalization
- Language Detection

- Entity Detection
- Filter Extraction
- Temporal Constraint
- Query Expansion
- Query Decomposition
- Search Strategy Selection

24.3 원본 Query 보존

변환된 Query와 원본 Query를 함께 기록한다.

24.4 Query Expansion

AI 기반 Query Expansion은 검색 범위를 넓힐 수 있지만 원본 의미와 다른 Query를 생성하지 않도록 제한한다.

25. Retrieval Engine

25.1 정의

Retrieval Engine은 Agent와 Workflow가 요청한 Knowledge를 정책 범위 안에서 검색한다.

25.2 기본 Retrieval 흐름

Query

→ Tenant Scope

→ Permission Filter

→ Collection Scope

→ Query Understanding

→ Metadata Filter

→ Keyword Search

→ Vector Search

→ Structured Search

- Graph Search
- Candidate Merge
- Ranking
- Re-ranking
- Quality Filter
- Citation Build
- Result

25.3 Retrieval Request

- Tenant ID
- Principal
- Agent ID
- Collections
- Query
- Filters
- Top K
- Minimum Score
- Retrieval Strategy
- Required Trust Level
- Required Freshness
- Citation Requirement
- Token Budget

25.4 Retrieval Result

- Knowledge ID
- Chunk ID
- Content

- Score
 - Rank
 - Source
 - Source Location
 - Version
 - Trust Level
 - Updated At
 - Citation
 - Metadata
-

26. Hybrid Search

26.1 정의

Hybrid Search는 여러 검색 방식을 결합하여 결과를 생성한다.

26.2 검색 방식

- Keyword
- BM25
- Vector Similarity
- Metadata
- Structured Query
- Graph Traversal
- Temporal Search

26.3 결합 방식

- Weighted Score
- Reciprocal Rank Fusion
- Rule-Based Merge

- Query-Type Routing

26.4 Query 유형별 전략

모든 Query에 Vector Search를 강제하지 않는다.

예:

- 정확한 Identifier → Keyword 또는 Structured Search
 - 의미 기반 질문 → Vector + Keyword
 - 관계 질문 → Graph Search
 - 기간 조건 → Metadata + Temporal Filter
-

27. Candidate Generation

27.1 정의

각 Search Engine에서 상위 후보를 생성한 후 공통 Candidate Set으로 병합한다.

27.2 Candidate Metadata

- Retrieval Method
- Raw Score
- Source
- Rank
- Collection
- Filter Match
- Trust
- Freshness

27.3 중복 Candidate

동일 Parent Content에서 유사 Chunk가 반복 등장하면 중복을 조정할 수 있다.

28. Ranking Engine

28.1 Ranking 요소

- Semantic Similarity
- Keyword Match
- Metadata Match
- Trust Level
- Freshness
- Source Priority
- Citation Quality
- User or Agent Scope
- Diversity

28.2 Ranking Policy

Collection 또는 Query 유형별 Ranking Weight를 정의할 수 있다.

28.3 신뢰도 반영

유사도가 높더라도 신뢰 수준이 낮은 Source는 상위로 자동 승격하지 않을 수 있다.

29. Re-ranking Engine

29.1 정의

Re-ranking은 1차 검색 결과를 보다 정밀한 모델 또는 규칙으로 다시 평가한다.

29.2 방식

- Cross Encoder
- Re-ranking Model
- LLM-Based Scoring
- Rule-Based
- Hybrid

29.3 Re-ranking 비용

고비용 Re-ranking은 모든 Query에 적용하지 않는다.

Query 중요도, 후보 수, 비용 정책에 따라 선택한다.

29.4 결과 기록

사용한 Re-ranker와 Score를 기록하여 Retrieval 재현성을 높인다.

30. Diversity and MMR

30.1 목적

상위 결과가 사실상 동일한 내용으로 채워지는 것을 방지한다.

30.2 적용 방식

- Maximum Marginal Relevance
- Source Diversity
- Section Diversity
- Time Diversity
- Entity Diversity

30.3 제한

Diversity 때문에 관련성이 현저히 낮은 결과를 포함해서는 안 된다.

31. Retrieval Filtering

31.1 필수 Filter 순서

Permission과 Tenant Filter는 검색 후가 아니라 가능한 한 검색 전에 적용한다.

31.2 Filter 항목

- Tenant
- Namespace
- Collection
- Resource Type

- Classification
- Trust Level
- Version
- Effective Date
- Language
- Tag
- Source
- Owner

31.3 Post-Filter 주의

Vector Store 특성상 Post-Filter가 필요한 경우 Unauthorized 결과가 Runtime Context에 노출되지 않도록 해야 한다.

32. Citation Engine

32.1 정의

Citation Engine은 Agent가 사용한 Knowledge Result를 원본 Source에 연결한다.

32.2 Citation 구조

- Citation ID
- Knowledge Resource ID
- Source ID
- Source Version
- Source Location
- Chunk ID
- Retrieved At
- Accessed By
- Retrieval Query

- Display Reference

32.3 원본 위치

가능한 경우 다음 위치를 제공한다.

- Page
- Section
- Paragraph
- Record
- Row
- Timestamp
- URI Reference

32.4 Citation 불가능한 Source

원본 위치를 특정할 수 없는 경우 Source-Level Citation임을 표시해야 한다.

32.5 Citation과 Model Output

Agent가 최종 답변을 생성할 때 사용된 Knowledge와 Citation을 연결할 수 있어야 한다.

33. Provenance

33.1 정의

Provenance는 Knowledge가 어떤 Source에서 어떤 처리 과정을 거쳐 생성되었는지를 나타낸다.

33.2 Provenance Chain

Original Source

→ Source Version

→ Parser Version

→ Normalization Version

→ Chunking Version

- Derived Resource
- Embedding Version
- Index Version
- Retrieval Result

33.3 필수 목적

Provenance는 다음에 필요하다.

- 오류 추적
- Source 확인
- Reindex
- 감사
- Version 재현
- 삭제 전파
- 품질 평가

34. Knowledge Versioning

34.1 Version 대상

- Source
- Document
- Knowledge Item
- Collection
- Metadata Schema
- Parser
- Chunking Policy
- Embedding Policy
- Retrieval Policy

- Index

34.2 Content Version

원본 Content 변경 시 신규 Version을 생성한다.

34.3 Metadata Revision

내용 변경 없이 Metadata만 수정된 경우 Revision으로 처리할 수 있다.

34.4 Historical Retrieval

정책에 따라 특정 시점 또는 Version 기준 Retrieval을 지원할 수 있다.

34.5 실행 Snapshot

Agent 또는 Workflow 실행에서 중요한 Knowledge 결과를 사용한 경우 당시 Source Version을 기록한다.

35. Freshness Manager

35.1 정의

Freshness Manager는 Knowledge가 현재 유효한지 평가한다.

35.2 Freshness 정보

- Source Updated At
- Ingested At
- Indexed At
- Effective From
- Effective Until
- Refresh Interval
- Maximum Staleness

35.3 Stale Knowledge

허용 Staleness를 초과한 Knowledge는 다음 상태가 될 수 있다.

- Warning

- Lower Rank
- Excluded
- Refresh Required

35.4 Query Freshness

Query가 최신성을 요구하는 경우 Freshness Constraint를 Retrieval Request에 포함한다.

36. Synchronization Manager

36.1 동기화 유형

- Manual
- Scheduled
- Event Driven
- Incremental
- Full Rebuild
- Near Real-Time

36.2 변경 처리

Source에서 다음 변경을 감지한다.

- Create
- Update
- Delete
- Move
- Permission Change
- Metadata Change

36.3 삭제 전파

Source에서 삭제된 항목은 정책에 따라 Knowledge와 Index에도 삭제 또는 비활성화 상태를 전파한다.

36.4 Permission 변경

원본 Source 권한이 변경된 경우 Retrieval 권한도 가능한 한 즉시 반영되어야 한다.

37. Structured Knowledge

37.1 정의

정형 데이터는 반드시 Text로 변환하여 Vector Search만 사용하는 방식으로 처리하지 않는다.

37.2 지원 방식

- SQL Adapter
- Query API
- Structured Retrieval
- Schema-Aware Search
- Aggregation
- Filter

37.3 Structured Query Policy

Agent가 생성한 Query는 실행 전에 다음을 검증해야 한다.

- Tenant Scope
- Allowed Table or Resource
- Allowed Operation
- Read or Write
- Query Cost
- Result Limit

37.4 Knowledge와 Operational Database

Knowledge Engine은 운영 Database를 무제한 Query하는 Gateway가 아니다.

허용된 Read Interface 또는 Snapshot을 사용해야 한다.

38. Semantic Layer

38.1 정의

서로 다른 Structured Source를 공통 의미 체계로 연결하기 위한 Semantic Layer를 둘 수 있다.

38.2 구성요소

- Concept
- Metric
- Dimension
- Entity
- Relationship
- Synonym
- Business Rule

38.3 적용 범위

Semantic Layer는 범용 구조만 제공하며 특정 산업의 용어를 Core에 포함하지 않는다.

39. Knowledge Package

39.1 정의

Knowledge Package는 배포 가능한 Knowledge 구성 단위이다.

39.2 Package 구성

- Manifest
- Knowledge Content 또는 Source Definition
- Metadata Schema
- Retrieval Policy
- Index Requirement

- Permission Requirement
- Version
- License
- Compatibility

39.3 Marketplace

Knowledge Package는 향후 Marketplace를 통해 배포할 수 있다.

39.4 Tenant 설치

공용 Package도 Tenant에 설치될 때 독립된 활성화 상태와 접근 정책을 가져야 한다.

40. Knowledge Access Control

40.1 Access 평가

Knowledge Retrieval은 최소한 다음을 평가한다.

- Principal
- Tenant
- Agent
- Collection
- Resource
- Classification
- Action
- Purpose
- Environment

40.2 Agent 접근

Agent는 Agent Definition에 명시된 Knowledge Scope만 조회할 수 있다.

40.3 Workflow 접근

Workflow가 Agent보다 넓은 Knowledge 권한을 가지고 있어도 Agent Step에 자동 전달

하지 않는다.

40.4 Document-Level Access

필요한 경우 Collection 단위뿐 아니라 Document 또는 Record 단위 권한을 지원해야 한다.

40.5 Field-Level Access

구조화 데이터는 정책에 따라 Field 또는 Column 단위 접근 제어를 지원할 수 있다.

41. Knowledge Security

41.1 기본 원칙

- Tenant Isolation
- Encryption
- Least Privilege
- Source Authentication
- Secure Connector
- Data Classification
- Secret Isolation
- Audit
- Retention
- Deletion

41.2 Prompt Injection 방어

외부 Knowledge Content는 신뢰할 수 없는 데이터로 취급한다.

Knowledge 내부의 명령문이 Agent System Instruction을 덮어쓸 수 없어야 한다.

41.3 악성 Content

수집된 파일과 Content는 다음 검사를 수행할 수 있다.

- Malware

- Embedded Script
- Dangerous File
- Invalid Link
- Unexpected Binary
- Data Exfiltration Pattern

41.4 외부 Model 전송

Embedding, Re-ranking 또는 LLM Processing을 위해 외부 Provider로 Knowledge를 전송하기 전에 Data Policy를 평가해야 한다.

41.5 Sensitive Result

Retrieval Result 자체도 원본과 동일한 Classification을 유지한다.

42. Knowledge Quality Engine

42.1 정의

Knowledge Quality Engine은 Knowledge가 Agent 판단에 사용할 수 있을 만큼 신뢰 가능한지 평가한다.

42.2 품질 기준

- Completeness
- Accuracy
- Freshness
- Provenance
- Duplication
- Consistency
- Parse Quality
- Metadata Completeness
- Retrieval Performance
- Citation Availability

42.3 Quality Score

Knowledge Resource 또는 Collection에 Quality Score를 부여할 수 있다.

42.4 자동 사용 제한

Quality Score가 기준 이하인 Resource는 검색에서 제외하거나 낮은 순위로 처리할 수 있다.

42.5 검증된 Knowledge

중요 Knowledge는 Human Review 또는 별도 검증 Workflow를 거쳐 Verified 상태로 전환할 수 있다.

43. Contradiction Management

43.1 정의

서로 다른 Source가 상충하는 정보를 제공할 수 있다.

43.2 처리 원칙

상충 정보를 무조건 하나로 병합하지 않는다.

각 Source와 Version을 독립적으로 유지한다.

43.3 판단 기준

Ranking 또는 Agent 판단 시 다음을 고려할 수 있다.

- Source Authority
- Freshness
- Verification State
- Effective Date
- Trust Level

43.4 Conflict 표시

중요한 상충이 탐지되면 Retrieval Result에 Conflict Metadata를 포함할 수 있다.

44. Knowledge Evaluation

44.1 Retrieval 평가

다음 지표를 사용할 수 있다.

- Recall@K
- Precision@K
- MRR
- NDCG
- Hit Rate
- Citation Accuracy
- Source Coverage

44.2 End-to-End 평가

Agent 결과까지 포함하여 다음을 평가할 수 있다.

- Groundedness
- Faithfulness
- Answer Relevance
- Citation Correctness
- Unsupported Claim Rate

44.3 Golden Dataset

핵심 Collection은 검증된 Query와 Expected Result Set을 유지할 수 있다.

44.4 Regression

Embedding, Chunking, Ranking 또는 Retrieval Policy 변경 시 기존 성능과 비교해야 한다.

45. Knowledge Lifecycle

45.1 Lifecycle 상태

Draft

- Ingesting
- Processing
- Validating
- Indexed
- Active
- Stale
- Suspended
- Deprecated
- Archived
- Deleted

45.2 Active

검색 및 Agent Context에 사용할 수 있는 상태이다.

45.3 Stale

유효성 확인 또는 Refresh가 필요한 상태이다.

45.4 Suspended

즉시 Retrieval에서 제외한다.

45.5 Archived

일반 검색에서는 제외하지만 역사적 조회 또는 Audit을 위해 보존할 수 있다.

46. Knowledge Deletion

46.1 삭제 유형

- Soft Delete
- Expiration
- Source Delete Propagation
- User Requested Delete

- Policy Delete
- Permanent Delete

46.2 삭제 전파 대상

Knowledge 삭제 시 다음 Derived Data도 처리해야 한다.

- Chunk
- Embedding
- Search Index
- Graph Node
- Relation
- Cache
- Generated Summary
- Read Model

46.3 Retrieval 차단 우선

완전 물리 삭제에 시간이 필요한 경우 먼저 Retrieval에서 즉시 제외해야 한다.

46.4 삭제 추적

삭제 작업은 Tombstone 또는 Audit 정보를 유지해야 한다.

47. Knowledge Retention

47.1 Retention Policy

다음 단위로 보존 정책을 설정할 수 있다.

- Source
- Collection
- Resource Type
- Classification
- Tenant

47.2 Expiration

Effective Until 또는 Retention 기간이 종료된 Resource는 자동 비활성화할 수 있다.

47.3 Legal Hold

삭제 금지가 필요한 경우 Legal Hold와 유사한 보존 Override를 지원할 수 있다.

이 기능도 명시적인 Governance와 Audit를 요구한다.

48. Knowledge Cache

48.1 Cache 대상

- Frequent Query
- Retrieval Result
- Metadata
- Access Decision
- Embedding Query
- Re-ranking Result

48.2 Cache Key

Cache Key에는 최소한 다음을 고려해야 한다.

- Tenant
- Query
- Collection
- Filter
- Policy Version
- Index Version

48.3 민감 데이터

민감한 Retrieval Result의 Cache 사용 여부는 Security Policy로 제어한다.

48.4 Cache는 Source of Truth가 아님

Cache 장애 또는 삭제가 Knowledge 원본에 영향을 주어서는 안 된다.

49. Knowledge API

49.1 주요 API 범주

- Source Management API
- Collection API
- Ingestion API
- Search API
- Retrieval API
- Index API
- Synchronization API
- Quality API
- Citation API
- Governance API

49.2 Retrieval API 입력

- Query
- Collection
- Filters
- Top K
- Retrieval Strategy
- Minimum Score
- Citation
- Freshness
- Trust Requirement

49.3 Retrieval API 출력

- Results
 - Score
 - Source
 - Citation
 - Metadata
 - Search Strategy
 - Index Version
 - Query ID
-

50. Agent와 Knowledge Engine 관계

50.1 Agent는 직접 Index를 조회하지 않는다.

Agent는 공식 Knowledge Retrieval Interface를 통해 검색한다.

50.2 Agent Query

Agent는 자신의 Knowledge Scope와 현재 Goal을 기준으로 Query를 생성한다.

50.3 Retrieval Context

Knowledge Engine은 Agent에 다음 정보만 전달한다.

- 허용된 Content
- Source
- Citation
- Metadata
- Trust Level
- Relevance

50.4 권한 우회 금지

Agent가 Query를 변경해도 Knowledge Scope를 확대할 수 없다.

50.5 Knowledge 활용 기록

Agent Execution에는 사용한 Knowledge Resource와 Version을 기록할 수 있어야 한다.

51. Workflow와 Knowledge Engine 관계

51.1 Knowledge Step

Workflow는 독립적인 Knowledge Retrieval Step을 사용할 수 있다.

51.2 Ingestion Workflow

Knowledge 수집과 갱신 자체를 Workflow Engine으로 운영할 수 있다.

51.3 Validation Workflow

새 Knowledge의 검증, 승인 및 Publish도 Workflow로 구성할 수 있다.

51.4 Workflow Context

Workflow 전체 Knowledge Context를 모든 Step에 자동 전달하지 않는다.

Step별 필요한 Scope만 전달한다.

52. Memory와 Knowledge의 경계

52.1 Knowledge

검증되고 재사용 가능한 정보 자산이다.

52.2 Memory

실행과 상호작용 과정에서 생성되는 맥락, 경험 및 상태이다.

52.3 Memory의 Knowledge 승격

Memory가 반복적으로 검증되고 공식적으로 재사용할 가치가 있다고 판단되더라도 자동으로 Knowledge가 되지 않는다.

다음 과정을 거쳐야 한다.

Memory

→ Candidate Knowledge

→ Validation

→ Provenance Check

→ Approval

→ Knowledge Resource

52.4 Knowledge의 Memory 복사

Agent가 Knowledge를 사용했다고 해서 전체 Knowledge를 Memory에 복제하지 않는다.
필요한 Reference 또는 요약만 저장한다.

53. Knowledge Observability

53.1 수집 대상

- Ingestion Count
- Ingestion Failure
- Parsing Failure
- Indexing Time
- Index Size
- Retrieval Count
- Retrieval Latency
- Zero Result Rate
- Average Score
- Re-ranking Latency
- Citation Rate
- Stale Resource Count
- Sync Delay
- Embedding Cost

53.2 Query Trace

하나의 Retrieval Query에 대해 다음을 추적할 수 있어야 한다.

Query Received

→ Scope Resolved

→ Query Rewritten

→ Keyword Search

→ Vector Search

→ Candidates Merged

→ Re-ranked

→ Policy Filtered

→ Citation Built

→ Result Returned

53.3 Result Debugging

운영자는 왜 특정 Resource가 검색 결과에 포함되었는지 확인할 수 있어야 한다.

예:

- Keyword Score
- Vector Score
- Re-rank Score
- Freshness Score
- Trust Score
- Final Score

54. Knowledge Audit

54.1 감사 대상

- Source 등록
- Connector 변경
- Collection 생성

- 권한 변경
- Classification 변경
- Knowledge Publish
- 중요 Knowledge 수정
- 삭제
- Export
- Restricted Retrieval
- 관리자 강제 변경
- Verification State 변경

54.2 일반 검색 Audit

모든 검색을 동일 수준으로 Audit할 필요는 없다.

데이터 등급과 정책에 따라 상세 수준을 조정한다.

55. Knowledge Cost Control

55.1 비용 항목

- Source Fetch
- Parsing
- OCR 또는 Media Processing
- Embedding
- Vector Storage
- Search
- Re-ranking
- Graph Processing
- Object Storage
- Network

55.2 비용 귀속

- Tenant
- Collection
- Source
- Ingestion Job
- Query
- Agent
- Workflow

55.3 비용 최적화

- Incremental Update
- Embedding Cache
- Batch Embedding
- Selective Re-ranking
- Index Tiering
- Archive Storage
- Duplicate Elimination

55.4 비용 때문에 보안을 낮추지 않음

비용 절감을 위해 민감 데이터를 허용되지 않은 외부 Provider에 전송해서는 안 된다.

56. Knowledge Scalability

56.1 Scale 대상

- Source 수
- Document 수
- Chunk 수
- Embedding 수

- Query Throughput
- Concurrent Ingestion
- Index Size
- Tenant 수

56.2 확장 원칙

Ingestion과 Retrieval을 독립적으로 확장할 수 있어야 한다.

56.3 Partition

필요한 경우 다음 기준으로 분할할 수 있다.

- Tenant
- Collection
- Region
- Time
- Data Classification

56.4 Hot과 Cold Knowledge

자주 조회되는 Knowledge와 장기 보존 Knowledge의 저장 계층을 분리할 수 있다.

57. Knowledge Provider Abstraction

57.1 교체 가능한 Provider

- Vector Database
- Search Engine
- Graph Database
- Embedding Provider
- Object Storage
- Parser Provider
- Re-ranking Provider

57.2 Adapter 원칙

Knowledge Domain Logic이 특정 Provider SDK에 직접 종속되지 않아야 한다.

57.3 Capability 기반 선택

Provider는 다음 Capability를 선언한다.

- Vector Filter
- Hybrid Search
- Metadata Filter
- Full Text Search
- Multi-Tenant Namespace
- Encryption
- Region
- Maximum Dimension
- Index Update Type

58. Knowledge Failure Handling

58.1 실패 유형

- Source Connection Failure
- Authentication Failure
- Parsing Failure
- Corrupted Data
- Embedding Failure
- Index Failure
- Search Provider Failure
- Re-ranking Failure
- Sync Failure

- Permission Sync Failure

58.2 처리 정책

- Retry
- Partial Success
- Quarantine
- Fallback Provider
- Previous Index 유지
- Alert
- Manual Review

58.3 검색 장애

Re-ranking 실패가 발생했다고 전체 Retrieval을 반드시 실패시키지 않는다.

정책에 따라 1차 검색 결과를 반환할 수 있다.

58.4 Index 구축 실패

신규 Index 구축 실패 시 기존 Active Index를 유지한다.

59. Knowledge Testing

59.1 Parser Test

각 Format의 구조와 Source Location 보존을 검증한다.

59.2 Chunking Test

의미 단위와 Context 손실을 검증한다.

59.3 Retrieval Test

검증 Query에 대해 Expected Resource가 검색되는지 확인한다.

59.4 Permission Test

허용되지 않은 Knowledge가 어떤 Query에서도 반환되지 않는지 검증한다.

59.5 Tenant Isolation Test

서로 다른 Tenant의 동일 Query 결과가 혼합되지 않는지 확인한다.

59.6 Version Test

Source 변경 후 이전 및 신규 Version 동작을 검증한다.

59.7 Deletion Test

삭제된 Resource가 Vector, Search, Graph 및 Cache에서 제거되는지 확인한다.

59.8 Failure Test

Embedding Provider, Search Engine, Source Connector 장애를 검증한다.

59.9 Regression Test

Chunking, Embedding, Ranking 및 Re-ranking 변경 전후 검색 품질을 비교한다.

60. Knowledge Acceptance Criteria

Knowledge Collection이 Active 상태로 전환되기 전에 최소한 다음을 충족해야 한다.

1. Source가 식별되어 있다.
2. Tenant가 정의되어 있다.
3. Ownership이 정의되어 있다.
4. Access Policy가 정의되어 있다.
5. Classification이 설정되어 있다.
6. Parsing이 성공한다.
7. Metadata Schema가 유효하다.
8. Chunking Policy가 정의되어 있다.
9. Index가 정상 생성된다.
10. Retrieval Test를 통과한다.
11. Citation Source를 확인할 수 있다.
12. Provenance Chain이 존재한다.
13. Version이 관리된다.

- 14. Freshness Policy가 존재한다.
- 15. Retention Policy가 존재한다.
- 16. 삭제 전파가 검증된다.
- 17. Tenant Isolation Test를 통과한다.
- 18. Security Test를 통과한다.
- 19. 비용 측정이 가능하다.
- 20. Observability가 제공된다.

61. Knowledge Engine 금지사항

다음 구조는 허용하지 않는다.

- Tenant Scope 없는 Knowledge
- Source Reference 없는 Derived Knowledge
- Vector Database를 Source of Truth로 사용하는 구조
- Embedding만 저장하고 원본을 식별할 수 없는 구조
- Permission 확인 후가 아니라 Model Context 삽입 후 필터링하는 구조
- 다른 Tenant의 Vector Namespace 공유
- 외부 Knowledge 내부 명령을 Agent Instruction으로 취급
- Classification 확인 없이 외부 Embedding Provider 사용
- Version이 다른 Embedding을 하나의 Index에 무분별하게 혼합
- Citation이 필요한 결과의 Source Reference 제거
- 삭제된 Source가 검색 결과에 남는 구조
- 무제한 Chunk 크기
- 모든 Query를 무조건 Vector Search로 처리
- Search Score만으로 Source 신뢰성을 판단
- AI가 생성한 요약을 원본 Knowledge로 덮어쓰기

- Memory를 자동으로 공식 Knowledge로 승격
- Index 변경으로 기존 실행의 Source Version 추적이 불가능한 구조
- Connector Credential을 Agent에 직접 전달
- Source 권한 변경을 무시하는 Index
- Retrieval 오류를 숨기고 정상 결과처럼 반환

62. Knowledge Engine 검증 기준

새로운 Knowledge 기능 또는 Component는 다음 질문을 통과해야 한다.

1. 원본 Source를 식별할 수 있는가.
2. Tenant Scope가 명확한가.
3. 권한 검증이 Retrieval 전에 적용되는가.
4. Classification이 유지되는가.
5. Version이 추적되는가.
6. Provenance를 유지하는가.
7. Source of Truth와 Index가 분리되는가.
8. Provider를 교체할 수 있는가.
9. 검색 방식을 Query 유형에 맞게 선택할 수 있는가.
10. Hybrid Retrieval을 지원할 수 있는가.
11. Source Citation이 가능한가.
12. Freshness를 판단할 수 있는가.
13. 중복과 상충을 관리할 수 있는가.
14. 삭제가 Derived Resource에 전파되는가.
15. Agent가 Knowledge Scope를 우회할 수 없는가.
16. 품질을 측정할 수 있는가.
17. 검색 결과를 재현·분석할 수 있는가.

18. 비용을 측정할 수 있는가.
19. 장애 시 이전 정상 상태를 유지할 수 있는가.
20. Product Vision, Agent Framework 및 Security 원칙에 부합하는가.

63. Knowledge Engine의 최종 정의

AGEX Knowledge Engine은 다양한 Source의 정보를 수집하고, 검증하고, 구조화하고, 버전 관리하고, 검색 가능한 형태로 변환하여 Agent와 Workflow에 통제된 방식으로 제공하는 AI Operating System의 공식 지식 계층이다.

Knowledge Engine은 문서를 Vector Database에 저장하는 단순 RAG 구성요소가 아니다.

원본 Source와 Provenance를 유지하면서 정형·비정형·그래프·벡터·Keyword Search를 통합하고, Query 특성에 따라 적절한 Retrieval 전략을 선택해야 한다.

모든 Knowledge는 Tenant, Ownership, Classification, Version, Trust Level, Access Policy, Freshness 및 Lifecycle을 가져야 한다.

Agent는 Knowledge Engine의 공식 Retrieval Interface를 통해서만 Knowledge에 접근하며, 자신의 Permission과 Knowledge Scope를 초과할 수 없다.

검색 결과는 가능한 한 Source와 Citation을 유지해야 하고, 사용된 Knowledge Version을 Agent 및 Workflow Execution과 연결할 수 있어야 한다.

Index와 Embedding은 언제든지 원본에서 다시 생성 가능한 Derived Resource이며 Source of Truth가 아니다.

AGEX Knowledge Engine은 다음 문장으로 최종 정의한다.

AGEX Knowledge Engine은 AI가 사용하는 정보를 단순 데이터가 아니라 출처·권한·버전·신뢰도·검색 근거가 관리되는 공식 지식 자산으로 전환하고, 필요한 순간에 검증 가능한 근거와 함께 제공하는 AI Operating System의 Knowledge Layer이다.

본 문서는 AGEX Knowledge의 생성, 관리, 검색, 품질, 보안, 버전 및 Lifecycle을 규정하는 공식 Knowledge Engine 문서로 사용한다.